

nvidia/NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4

like
 54

Follow
 NVIDIA
 59k

Text Generation

Transformers

Safetensors

PyTorch

nvidia/nemotron-post-training-v3

nvidia/nemotron-pre-training-datasets

12 languages

nvidia

nemotron-3

latent-moe

mtp

conversational

8-bit precision

License: openmdw-1.1

⋮

Deploy

▼

Copy to bucket

NEW

Use this model

▼

Model card

Files

xet

Community
 3

Configuration Parsing Warning:Config file config.json cannot be fetched (too big)

NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4

Chat

Paper

Read Now!

Pre-Training Datasets

Available Here

Post-Training Datasets

Available Here

Nemotron Developer Page

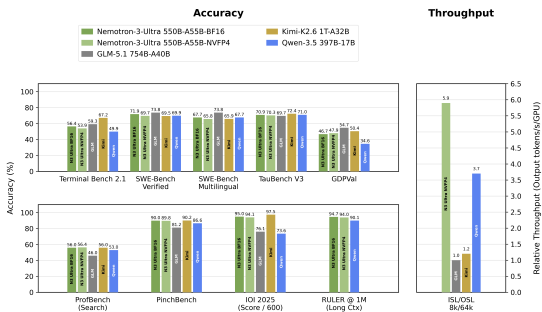
Learn More Here!

Discord

NVIDIA AI Developer

License

OpenMDW-1.1



Model Summary

Total Parameters	550B (55B active)
Architecture	LatentMoE - Mamba-2 + MoE + Attention hybrid with Multi-Token Prediction (MTP)
Context Length	Up to 1M tokens
Minimum GPU Requirement	4xGB200, 4xB200, 4x GB300, 4x B300, 8xH100



Safetensors ⓘ

Model size
 335B params

Tensor type
 F32 · BF16 · F8_E4M3 · U8

Chat template

Inference Providers

Text Generation

Run 15,000+ Models Instantly

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

nvidia/NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4 is supported by the following Inference Providers:

Together AI

[View API Code](#)
[Dismiss](#)

Your prompt here...

Send

View Code
 Compare providers

Model tree for nvidia/NVIDIA-Nem...

Finetunes
 1 model

Space using nvidia/NVIDIA-Nem... 1

akhalq/NVIDIA-Nemotron-3-Ultra-550...

Collections including nvidia/NVIDI...

Inference Optimized C...
 Collection

A collectio...
 • 74 items
 • U...
 • 165

Supported Languages	English, French, Spanish, Italian, German, Japanese, Korean, Hindi, Korean, Brazilian Portuguese, and Chinese
Best For	Frontier reasoning, complex agentic workflows, long-context analysis, tool use, multilingual reasoning, high-stakes RAG
Reasoning Mode	Configurable on/off via chat template (enable_thinking=True/False)
License	OpenMDW License Agreement, version 1.1
Release Date	June 4, 2026

NVIDIA Nemotron v3  Collection

Open, P... • 23 items • Upd... •  310

Quick Start

For more details on how to deploy and use the model - see the [Quick Start Guide](#) below!

Model Overview

Model Developer: NVIDIA Corporation

Model Dates: December 2025 - April 2026

Data Freshness:

- The post-training data has a cutoff date of May 2026.
- The pre-training data has a cutoff date of September 2025.

What is Nemotron?

NVIDIA Nemotron™ is a family of open models with open weights, training data, and recipes, delivering leading efficiency and accuracy for building specialized AI agents.

Description

Nemotron-3-Ultra-550B-A55B-NVFP4 is a frontier-scale large language model (LLM) trained by NVIDIA, designed to deliver strong agentic, reasoning, and conversational capabilities. It is optimized for the most demanding workloads, including complex multi-step agents, long-context

analysis, and high-accuracy reasoning over code, math, and science. Like other models in the family, it responds to user queries and tasks by first generating a reasoning trace and then concluding with a final response. The model's reasoning capabilities can be configured through a flag in the chat template.

The model employs a hybrid **Latent Mixture-of-Experts (LatentMoE)** architecture, utilizing interleaved Mamba-2 and MoE layers, along with select Attention layers. Like the Super model, the Ultra model incorporates **Multi-Token Prediction (MTP)** layers for faster text generation and improved quality, and it is trained using an **NVFP4** pre-training recipe to maximize compute efficiency. The model has **55B active parameters** and **550B parameters in total**.

The supported languages include: English, French, Spanish, Italian, German, Japanese, Korean, Hindi, Korean, Brazilian Portuguese, and Chinese.

This model is ready for commercial and non-commercial use.

License/Terms of Use

Governing Download Terms: Use of this model is governed by the [OpenMDW-1.1 model license](#).

Benchmarks

Benchmark	Nemotron 3 Ultra BF16	Nemotron 3 Ultra NVFP4
Agentic		
Terminal Bench 2.1	56.4	53.9
GDPVal	46.7	47.9
SWE-Bench Verified	71.9	69.7
SWE-Bench Multilingual	67.7	65.8
ProfBench (Search)	56	56.4
PinchBench	90	89.8
TauBench V3		

Benchmark	Nemotron 3	Nemotron 3
	Ultra BF16	Ultra NVFP4
Airline	81.5	80.0
Retail	86.4	88.4
Telecom	92.9	93.6
Banking	22.6	19.2
Average	70.9	70.3
BrowseComp	44.4	41.4
Reasoning and Knowledge		
IOI 2025	570.0	564.7
GPQA (no tools)	87.0	87.9
SciCode (subtask)	44.6	43.5
HLE (no tools)	26.7	26.1
CritPt (no tools)	3.1	3.4
OmniScience Accuracy	24.1	24.6
OmniScience Non-Hallucination	78.7	75.5
Chat & Instruction Following		
IFBench (prompt)	81.7	82.3
Long Context		
AA-LCR	65.4	65.5
RULER 1M	94.7	94.0

All evaluation results were collected via [Nemo Evaluator SDK](#). We used three main evaluation harnesses: [Nemo Gym](#), [Nemo Skills](#), and [Harbor](#) with extended sandboxing support via AWS ECS on Nemo Evaluator. In addition, the evaluations also used dedicated open-source packaged containers for ScaleAI Multi Challenge Multi Turn Instruction Following and KernelBench. For reproducibility purposes, more details on the evaluation settings and pinned containers can be found in the [Nemo Evaluator SDK examples folder](#) and the [reproducibility tutorial for Nemotron 3 Ultra](#).

The following benchmarks are not onboarded yet in our open source tools and for these we used either their official open source implementation or otherwise an internal scaffolding that we plan to open source in the future: BrowseComp with Search, Tau Bench 3, ProfBench with Search, PinchBench, Vals.ai, LongBench v2.

Deployment Geography: Global

Use Case

NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4 is a frontier-scale general purpose reasoning and chat model intended to be used in English, Code, and supported multilingual contexts. This model is optimized for complex agentic workflows, long-context reasoning, and high-stakes analytical workloads. It is intended to be used by developers designing AI Agent systems, chatbots, RAG systems, and other AI-powered applications. This model is also suitable for complex instruction-following tasks and long-context reasoning over very large documents and codebases.

Release Date

Hugging Face - 06/04/2026 via [Hugging Face](#)

Reference(s)

- [NVIDIA Nemotron 3 model family on Hugging Face](#)
- [NVIDIA Nemotron 3 Ultra Technical Report](#)

Model Architecture

- **Architecture Type:** Mamba2-Transformer Hybrid Latent Mixture of Experts (LatentMoE) with Multi-Token Prediction (MTP)
- **Network Architecture:** Nemotron Hybrid LatentMoE
- **Number of model parameters:** 550B Total / 55B Active

Model Design

The model utilizes the **LatentMoE** architecture, where tokens are projected into a smaller latent

dimension for expert routing and computation, improving accuracy per byte. The Ultra model is pre-trained using an NVFP4 recipe — sharing the quantization-aware pre-training approach pioneered in the Nemotron 3 family. The majority of linear layers use NVFP4 for weights, activations, and gradients, while select layers (including latent projections, MTP layers, QKV/attention projections, and embeddings) are maintained in BF16 or MXFP8 for training stability. The model includes **Multi-Token Prediction (MTP)** layers using a shared-weight design across prediction heads. This improves training signal quality, enables faster inference via native speculative decoding, and supports more stable autoregressive drafting at longer draft lengths compared to independently trained offset heads.

Training Methodology

Stage 1: Pre-Training

- [NVIDIA-Nemotron-3-Ultra-550B-A55B-Base-BF16](#) model was pre-trained for approximately 20T tokens using crawled and synthetic code, math, science, and general knowledge data. Training leveraged an NVFP4 recipe for efficiency. All datasets are disclosed in the [Training and Evaluation Datasets](#) section of this document. Major portions of the pre-training corpus are released in the [Nemotron-Pre-Training-Datasets](#) collection.
- Software used for pre-training: [Megatron-LM](#)

Stage 2: Supervised Fine-Tuning

- The model was further fine-tuned on synthetic code, math, science, tool calling, instruction following, structured outputs, and general knowledge data. This stage incorporated data designed to support long-range retrieval and multi-document aggregation. All datasets are disclosed in the [Training and Evaluation Datasets](#) section of this document. Major portions of the fine-tuning corpus are released in the [Nemotron-Post-Training-v3](#) collection. [Data Designer](#) is one of the libraries used to prepare these corpora.

Stage 3: Reinforcement Learning

- The model underwent multi-environment reinforcement learning using asynchronous GRPO (Group Relative Policy Optimization) across math, code, science, instruction following, multi-step tool use, multi-turn conversations, and structured output environments. It utilized an asynchronous RL architecture that fully decouples training from inference across separate GPU devices, leveraging in-flight weight updates and MTP to accelerate rollout generation. Conversational quality was further refined through RLHF. All datasets are disclosed in the [Training and Evaluation Datasets](#) section of this document. The RL environments and datasets are released as part of [NeMo Gym](#).
- Software used for reinforcement learning: [NeMo RL](#), [NeMo Gym](#)

Stage 4: Multi-Domain On-Policy Distillation (MOPD)

- The model underwent **Multi-Domain On-Policy Distillation (MOPD)** to improve reasoning across many task types while staying efficient. This technique uses strong teacher models to guide training on the model's own generated attempts (on-policy rollouts), helping recover accuracy and improve performance across coding, math, instruction following, tool use, and agentic workflows. By distilling teacher signal onto the student's own trajectories rather than offline traces, MOPD better aligns the student's behavior with what it would actually produce at inference time, yielding stronger gains than purely off-policy distillation.

NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4 model is a result of the above work.

The end-to-end training recipe is available in the [NVIDIA Nemotron Developer Repository](#). Evaluation results can be replicated using the [NeMo Evaluator SDK](#). [Data Designer](#) is one of the libraries used to prepare the pre and post training datasets. More details on the datasets and synthetic

data generation methods can be found in the technical report [NVIDIA Nemotron 3 Ultra Technical Report](#).

Input

- **Input Type(s):** Text
- **Input Format(s):** String
- **Input Parameters:** One-Dimensional (1D): Sequences
- **Other Properties Related to Input:** Maximum context length up to 1M tokens. Supported languages include: English, French, Spanish, Italian, German, Japanese, Korean, Hindi, Korean, Brazilian Portuguese, and Chinese.

Output

- **Output Type(s):** Text
- **Output Format:** String
- **Output Parameters:** One-Dimensional (1D): Sequences
- **Other Properties Related to Output:** Maximum context length up to 1M tokens

Our AI models are designed and optimized to run on NVIDIA GPU-accelerated systems. By leveraging NVIDIA's hardware (e.g. GPU cores) and software frameworks (e.g., CUDA libraries), the model achieves faster training and inference times compared to CPU-only solutions.

Software Integration

- Runtime Engine(s): NeMo 26.04.01
- Supported Hardware Microarchitecture
Compatibility: NVIDIA Ampere - A100; NVIDIA Blackwell; NVIDIA Hopper - H100-80GB
- Operating System(s): Linux

The integration of foundation and fine-tuned models into AI systems requires additional testing using use-case-specific data to ensure safe and effective deployment. Following the V-model methodology, iterative testing and validation at both unit and system levels are essential to mitigate risks, meet technical and functional

requirements, and ensure compliance with safety and ethical standards before deployment.

Model Version(s)

- v1.0 - GA

Quick Start Guide

The Ultra NVFP4 checkpoint is a frontier-scale model quantized for maximum throughput on the latest hardware. The minimum recommended hardware is:

- **Single-node:** 4× B200 (fits NVFP4 weights plus KV cache with headroom)
- **Multi-node:** ≥4 GPUs across GB200 / GB300

All deployment snippets below default to port 8000, with chunked prefill, NVFP4 KV caching, and MTP (5 speculative tokens) enabled.

Multi-Node Setup with Ray (Recommended for multi-node deployments)

The recommended multi-processing backend for multi-node deployments is Ray v2. Below is a template for launching a Ray cluster:

```
# Set the IP for the head node in RAY_HEAD_IP
export RAY_HEAD_IP=<head_node_ip>
export RAY_PORT=6379
export RAY_ADDRESS=${RAY_HEAD_IP}:${RAY_PORT}

# Start Ray head node (vLLM/SGLang will run on this node)
ray start --head --node-ip-address=${RAY_ADDRESS}

# Start Ray worker node(s)
ray start --address=${RAY_HEAD_IP}:${RAY_PORT}

# Verify Ray cluster is ready
ray status --address=${RAY_HEAD_IP}:${RAY_PORT}
```

Note: ray[cgraph] is required: `uv pip install "ray[cgraph]"`

vLLM

Recommended container: vllm/vllm-openai:v0.22.0

For more detailed information, please see this [cookbook](#).

```
export MODEL_CKPT=nvidia/NVIDIA-Nemotron-3-U
```

4× B200 single-node deployment5:

```
docker run -d --name nemotron-ultra-vllm \
  --gpus all \
  --ipc=host \
  --network=host \
  --shm-size=16g \
  --ulimit memlock=-1 \
  --ulimit stack=67108864 \
  -v $MODEL_CKPT:/model:ro \
  -e VLLM_WORKER_MULTIPROC_METHOD=spawn \
  -e SAFETENSORS_FAST_GPU=1 \
  -e NVIDIA_TF32_OVERRIDE=1 \
  -e VLLM_LOGGING_LEVEL=INFO \
  vllm/vllm-openai:v0.22.0 \
  /model \
  --host 0.0.0.0 \
  --port 8000 \
  --served-model-name nvidia/nemotron-3-ultra \
  --trust-remote-code \
  --tensor-parallel-size 4 \
  --enable-expert-parallel \
  --kv-cache-dtype fp8 \
  --max-model-len 262144 \
  --gpu-memory-utilization 0.90 \
  --max-num-seqs 16 \
  --max-num-batched-tokens 32768 \
  --enable-chunked-prefill \
  --enable-prefix-caching \
  --reasoning-parser nemotron_v3 \
  --enable-auto-tool-choice \
  --tool-call-parser qwen3_coder \
  --mamba-ssm-cache-dtype float16 \
  --mamba-backend flashinfer \
  --enable-mamba-cache-stochastic-rounding \
  --mamba-cache-philox-rounds 5 \
  --speculative-config '{"method": "nemotron \
  --model-loader-extra-config '{"enable_mult:
```

Multi-node deployment (e.g. 2× 4×GB300 with

Ray): After launching the Ray head and worker per the multi-node setup above:

```
# Run on Ray head node
vllm serve $MODEL_CKPT \
  --host 0.0.0.0 \
  --port 8000 \
  --served-model-name nvidia/nemotron-3-ultra
```

```

--tensor-parallel-size 4 \
--pipeline-parallel-size 2 \
--distributed-executor-backend ray \
--trust-remote-code \
--kv-cache-dtype fp8 \
--gpu-memory-utilization 0.90 \
--max-model-len 262144 \
--max-num-seqs 256 \
--max-num-batched-tokens 32768 \
--enable-chunked-prefill \
--enable-prefix-caching \
--reasoning-parser nemotron_v3 \
--enable-auto-tool-choice \
--tool-call-parser qwen3_coder \
--mamba-ssm-cache-dtype float16 \
--mamba-backend flashinfer \
--enable-mamba-cache-stochastic-rounding \
--mamba-cache-philox-rounds 5 \
--speculative-config '{"method": "nemotron_
--model-loader-extra-config '{"enable_mult:
--compilation-config '{"pass_config": {"fu
--distributed-timeout-seconds 3600

```

- **Context Length:** Defaults to 256k above. To use up to 1M, set `VLLM_ALLOW_LONG_MAX_MODEL_LEN=1` and `--max-model-len 1048576`.
- **Useful environment variables:**
`VLLM_FLASHINFER_ALLREDUCE_BACKEND=trtllm`,
`VLLM_FLASHINFER_MOE_BACKEND=latency`
(TRTLLM-Gen) or
`VLLM_FLASHINFER_MOE_BACKEND=throughput`
(CUTLASS).

SGLang

Container (tested on 4× B200): `docker pull lmsysorg/sglang:v0.5.12.post1`

For more detailed information, please see this [cookbook](#).

4× B200 single-node deployment (NVFP4):

```

docker run -d --name nemotron-ultra-sglang \
  --gpus all \
  --cap-add SYS_NICE \
  --ipc=host \
  --network=host \
  --shm-size=16g \
  --ulimit memlock=-1 \
  --ulimit stack=67108864 \
  -v $MODEL_CKPT:/model:ro \

```

```

-e SAFETENSORS_FAST_GPU=1 \
-e NVIDIA_TF32_OVERRIDE=1 \
-e SGLANG_DISABLE_DEEP_GEMM=1 \
lmsysorg/sglang:v0.5.12.post1 \
python3 -m sglang.launch_server \
--model-path /model \
--host 0.0.0.0 \
--port 8000 \
--served-model-name nvidia/nemotron-3-ultra \
--tp-size 4 \
--ep-size 4 \
--context-length 262144 \
--mem-fraction-static 0.85 \
--chunked-prefill-size 32768 \
--fp8-gemm-backend triton \
--moe-runner-backend triton \
--mamba-scheduler-strategy no_buffer \
--disable-piecewise-cuda-graph \
--reasoning-parser nemotron_3 \
--tool-call-parser qwen3_coder \
--speculative-algorithm EAGLE \
--speculative-num-steps 5 \
--speculative-eagle-topk 1 \
--speculative-num-draft-tokens 5 \
--kv-cache-dtype fp8 \
--trust-remote-code \
--log-level info

```

- **Context length:** Defaults to 256k above. To use up to 1M, set `SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1` and `--context-length 1048576`.
- **Tool calls + reasoning parsing:** When calling the chat completions endpoint with tools, you must set `"chat_template_kwargs": {"enable_thinking": true, "force_nonempty_content": true}` in the request body to parse both reasoning and tool calls correctly.

TRT-LLM

Container: `docker pull nvcr.io/nvidia/tensorrt-llm/release:1.3.0rc17`

For more detailed information, please see this [cookbook](#).

Max-throughput configuration (B200×4):

```

cat > ./extra-llm-api-config.yml << EOF
cuda_graph_config:

```

```

enable_padding: true
max_batch_size: 16

enable_chunked_prefill: true
enable_attention_dp: true
max_num_tokens: 16384
num_postprocess_workers: 4
stream_interval: 10

kv_cache_config:
  dtype: fp8
  enable_block_reuse: false
  free_gpu_memory_fraction: 0.7
  mamba_ssm_cache_dtype: float16
  mamba_ssm_philox_rounds: 5
  mamba_ssm_stochastic_rounding: true
  mamba_state_cache_interval: 16384

moe_config:
  backend: CUTEDSL
  max_num_tokens: 65536
  use_low_precision_moe_combine: true

speculative_config:
  decoding_type: MTP
  max_draft_len: 3
  max_total_draft_tokens: 3
  num_nextn_predict_layers: 3
  speculative_model: null
  allow_advanced_sampling: true
EOF

```

```

trtllm-serve $MODEL_CKPT \
  --backend pytorch \
  --host 0.0.0.0 \
  --port 8000 \
  --max_batch_size 16 \
  --tp_size 4 --ep_size 4 \
  --max_num_tokens 16384 \
  --trust_remote_code \
  --reasoning_parser nano-v3 \
  --tool_parser qwen3_coder \
  --chat_template $MODEL_CKPT/chat_template.:
  --extra_llm_api_options extra-llm-api-conf:

```

Min-latency configuration (B200×4, NVFP4):

```

cat > ./extra-llm-api-config.yml << EOF
backend: pytorch
trust_remote_code: true
tensor_parallel_size: 4
pipeline_parallel_size: 1
context_parallel_size: 1
gpus_per_node: 8
moe_expert_parallel_size: 1

```

```

disable_overlap_scheduler: false

cuda_graph_config:
  batch_sizes:
    max_batch_size: 8
    enable_padding: true

enable_chunked_prefill: true
enable_attention_dp: false
max_batch_size: 8
max_seq_len: null
max_num_tokens: 8192
num_postprocess_workers: 0

kv_cache_config:
  dtype: fp8
  free_gpu_memory_fraction: 0.75
  mamba_state_cache_interval: 8192

moe_config:
  backend: TRTLLM

speculative_config:
  decoding_type: MTP
  max_draft_len: 5
  max_total_draft_tokens: 5
  # rc14 compatibility: rc14 derives max_dra:
  # rc15+ keeps max_draft_len and ignores th:
  num_nextn_predict_layers: 5
  allow_advanced_sampling: true
EOF

```

```

trtllm-serve \
<nvfp4_ckpt> \
--max_batch_size 8 \
--tp_size 4 --ep_size 1 \
--max_num_tokens 16384 \
--trust_remote_code \
--reasoning_parser nano-v3 \
--tool_parser qwen3_coder \
--chat_template $MODEL_DIR/chat_template.jin:
--extra_llm_api_options extra-llm-api-config

```

Long-context configuration: For long-context benchmarking, set `TLLM_ALLOW_LONG_MAX_MODEL_LEN=1` as an environment variable and add `--max_seq_len <seq_len>` as the desired maximum context length. The MTP speculative_config block above carries over unchanged — on rc16, `max_draft_len` is the authoritative field and `num_nextn_predict_layers` is treated as deprecated.

API Client

The examples below use the OpenAI-compatible client and work with any of the serving backends above.

NOTE: For coding agents add the following to the API call - `extra_body={"chat_template_kwargs": {"force_nonempty_content": True}}`

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8080")
MODEL = "nvidia/nemotron-3-ultra"
```

Reasoning ON (default)

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "What is the capital of France?"}],
    max_tokens=16000,
    temperature=1.0,
    top_p=0.95,
    extra_body={"chat_template_kwargs": {"enable_reasoning": True}}
)
print(response.choices.message.content)
```

Reasoning OFF

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "What is the capital of France?"}],
    max_tokens=16000,
    temperature=1.0,
    top_p=0.95,
    extra_body={"chat_template_kwargs": {"enable_reasoning": False}}
)
print(response.choices.message.content)
```

Medium-effort reasoning

Uses significantly fewer reasoning tokens than full thinking mode. Recommended as a starting point before tuning explicit token budgets.

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "What is the capital of France?"}],
    max_tokens=16000,
    temperature=1.0,
    top_p=0.95,
    extra_body={"chat_template_kwargs": {"enable_reasoning": True, "reasoning_effort": "medium"}}
)
print(response.choices[0].message.content)
```

Tool calling with reasoning (SGLang requires explicit chat template kwargs)

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "What's the weather in New York?"}],
    tools=[{
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the current weather in a specific city",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "The city to get the weather for"},
                    "unit": {"type": "string", "description": "The unit of temperature (Celsius or Fahrenheit)"}
                },
                "required": ["city"]
            }
        }
    }],
    tool_choice="required",
    max_tokens=256,
    temperature=1.0,
    top_p=0.95,
    extra_body={"chat_template_kwargs": {"enable_tool_calls": True}}
)
```

OpenCode

OpenCode is an AI coding agent that runs in your terminal. It connects to any OpenAI-compatible endpoint, making it compatible with all three serving backends above (vLLM, SGLang, and TRT-LLM).

Create or update your

~/.config/opencode/opencode.json:

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "local/nvidia-nemotron-3-ultra",
  "provider": {
    "local": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "local_backend",
      "options": {
        "baseUrl": "http://localhost:8000",
        "apiKey": "EMPTY"
      }
    },
    "models": {
      "nvidia-nemotron-3-ultra": {
        "name": "nvidia/nemotron-3-ultra"
      }
    }
  }
}
```



```

        "limit": {
            "context": 1000000,
            "output": 32768
        }
    },
    "agent": {
        "build": {
            "temperature": 1.0,
            "top_p": 0.95,
            "max_tokens": 32000
        },
        "plan": {
            "temperature": 1.0,
            "top_p": 0.95,
            "max_tokens": 32000
        }
    }
}

```

All backends above default to port 8000, so the `baseUrl` works as-is for vLLM, SGLang, and TRT-LLM. To learn more about other supported agent scaffolds, check out [this resource](#).

Set a hard token ceiling on the reasoning trace using `reasoning_budget`. The model will attempt to close the trace at the next newline before the budget is hit; if none is found within 500 tokens it closes abruptly at `reasoning_budget + 500`.

```

from typing import Any, Dict, List
import openai
from transformers import AutoTokenizer

class ThinkingBudgetClient:
    def __init__(self, base_url: str, api_key: str):
        self.tokenizer = AutoTokenizer.from_pretrained('gpt-4o')
        self.client = openai.OpenAI(base_url=base_url, api_key=api_key)

    def chat_completion(
        self,
        model: str,
        messages: List[Dict[str, Any]],
        reasoning_budget: int = 512,
        max_tokens: int = 1024,
        **kwargs,
    ) -> Dict[str, Any]:
        assert max_tokens > reasoning_budget
        if reasoning_budget > 1000:
            raise ValueError(f"reasoning_budget must be less than 1000")
        if max_tokens > 1000:
            raise ValueError(f"Got {max_tokens=} and {reasoning_budget=} which is too high")

```

```

    )

    # Step 1: generate the reasoning trace
    response = self.client.chat.completions.create(
        model=model, messages=messages, max_tokens=max_tokens
    )
    reasoning_content = response.choices[0].text
    if "</think>" not in reasoning_content:
        reasoning_content = f"{reasoning_content}</think>"

    reasoning_tokens_len = len(
        self.tokenizer.encode(reasoning_content)
    )
    remaining_tokens = max_tokens - reasoning_tokens_len
    assert remaining_tokens > 0, (
        f"No tokens remaining for response generation. "
        "Increase max_tokens or lower reasoning budget."
    )

    # Step 2: continue from the closed reasoning trace
    messages.append({"role": "assistant", "content": reasoning_content})
    prompt = self.tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=True
    )
    response = self.client.completions.create(
        model=model, prompt=prompt, max_tokens=max_tokens
    )

    return {
        "reasoning_content": reasoning_content,
        "content": response.choices[0].text,
        "finish_reason": response.choices[0].finish_reason
    }

# Example usage (32-token reasoning budget):
client = ThinkingBudgetClient(
    base_url="http://localhost:8000/v1",
    api_key="EMPTY",
    tokenizer_name_or_path="nvidia/NVIDIA-Nemotron-3-Ultra-550B"
)

result = client.chat_completion(
    model="nvidia/NVIDIA-Nemotron-3-Ultra-550B",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"}
    ],
    reasoning_budget=32,
    max_tokens=512,
    temperature=1.0,
    top_p=0.95,
)
print(result)

```

Training

Data Modality: Text

The total size: 53.8 TiB (14.8 trillion tokens)

Total number of datasets: 226

Dataset partition: *Training [100%], testing [0%], validation [0%]*

Time period for training data collection: 2013 to 2026

Time period for testing data collection: 2013 to 2026

Time period for validation data collection: 2013 to 2026

Data Collection Method by dataset: Hybrid:

Automated, Human, Synthetic

Labeling Method by dataset: Hybrid: Automated, Human, Synthetic

NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4 is pre-trained on a large corpus of high-quality curated and synthetically-generated data. It is trained in the English language, as well as 11 other languages and 43 programming languages. Our sources cover a variety of document types such as: webpages, dialogue, articles, and other written materials. The corpus spans domains including legal, math, science, finance, and more. We also include a small portion of question-answering, and alignment style data to improve model accuracy. The model was pre-trained for approximately 20 trillion tokens.

The post-training corpus for NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4 consists of high-quality curated and synthetically-generated data. Primary languages used for post-training include English, French, Spanish, Italian, German, Japanese, Korean, Hindi, Korean, Brazilian Portuguese, and Chinese

These datasets, such as FinePDFs, EssentialWeb, HotpotQA, SQuAD, and HelpSteer3, do not collectively or exhaustively represent all demographic groups (and proportionally therein). For instance, these datasets do not contain explicit mentions of demographic classes such as age, gender, or ethnicity in 64-99% of samples, depending on the source. In the subset where such terms are present, document-based datasets

(FinePDFs and EssentialWeb) contain representational skews, such as references to "male" outnumbering those to "female", and mentions of "White" as the most frequent among ethnic identifiers (comprising 43-44% of ethnicity mentions). To mitigate these imbalances, we recommend considering evaluation techniques such as bias audits, fine-tuning with demographically balanced datasets, and mitigation strategies like counterfactual data augmentation to align with the desired model behavior. This evaluation used a 3,000-sample subset per dataset, identified as the optimal threshold for maximizing embedder accuracy.

During post-training, we generate synthetic data by distilling trajectories, solutions, and translations from strong teacher models and agent systems, often grounded in real tasks or documents and aggressively filtered for quality. For math, code, and science, we start from curated problem sets and use open source permissive models such as GPT-OSS-120B to produce step-by-step reasoning traces, candidate solutions, best-of-n selection traces, and verified CUDA kernels. For long-context and science, we build synthetic QA and reasoning data by retrieving passages from long documents, generating MCQ/OpenQA questions and answers, and paraphrasing them into multiple prompt/response formats to ensure diversity. Across all pipelines we stack automated verification—compilers, numerical checks, language identification—to ensure our data is high quality.

For all domains, we apply a unified data filtering pipeline to ensure that only high-quality, license-compliant, and verifiable samples are used for post-training. We first discard malformed examples using structural checks (e.g., missing tool definitions when tool calls are present). We then aggressively filter reasoning traces exhibiting pathological repetition, such as repeated n-grams within a sliding window or across the entire trajectory, which we found to be a strong indicator of malformed or low-quality reasoning. Finally, based on internal audits of synthetically generated datasets, we observed that some teacher models occasionally produce reasoning traces and final responses that implicitly align with specific

political entities or promote nationalistic narratives. To mitigate this, we apply targeted keyword- and regex-based filters and remove all trajectories matching such behavior.

Alongside the model, we release our final pre-training and post-training data, as outlined in this section. For ease of analysis, there is a sample set that is ungated. For all remaining code, math and multilingual data, gating and approval is required, and the dataset is permissively licensed for model training purposes.

More details on the datasets and synthetic data generation methods can be found in the technical report [***NVIDIA Nemotron 3 Ultra***](#).

► For Detailed Dataset Information: [Click here!](#)

Inference

- **Acceleration Engine:** PyTorch
- **Test Hardware:**
 - NVIDIA Hopper
 - H100
 - H200
 - NVIDIA Grace Blackwell
 - GB200
 - GB300
 - NVIDIA Blackwell
 - B200
 - B300

Ethical Considerations

NVIDIA believes Trustworthy AI is a shared responsibility and we have established policies and practices to enable development for a wide array of AI applications. When downloaded or used in accordance with our terms of service, developers should work with their internal model team to ensure this model meets requirements for the relevant industry and use case and addresses unforeseen product misuse.

We advise against circumvention of any provided safety guardrails contained in the Model without a substantially similar guardrail appropriate for your use case. For more details: [Safety](#) and [Explainability](#) Subcards.

For more detailed information on ethical considerations for this model, please see the Model Card++ [Bias](#), and [Privacy](#) Subcards.

Please report model quality, risk, security vulnerabilities or NVIDIA AI Concerns [here](#).

Citation

System theme

TOS

Privacy

About

Careers



Models

Datasets

Spaces

Pricing

Docs

```
@misc{nvidia_nemotron_3_ultra_2026,
  title = {Nemotron 3 Ultra: Open, Efficient, and Scalable},
  author = {{NVIDIA}},
  year = {2026},
  url = {https://research.nvidia.com/labs,
  note = {White Paper}
}
```